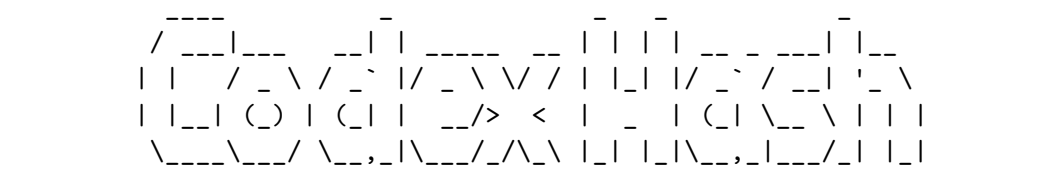

~ Codex Hash Research Laboratory · Whitepaper Series · 2026

Graceful Refusals as Silent Successes: A Pre-Registered Protocol for Characterising `claude -p` Exit-Code Semantics

Author · Ulisses Flores CTO & Chief Researcher · Codex Hash Research Laboratory · São Paulo, Brazil MSc Program in Artificial Intelligence (in progress) · American Global Tech University c.ulisses@gmail.com · https://ulissesflores.com

Whitepaper · **Version 1.0** · **2026-05-18 DOI** https://doi.org/10.5281/zenodo.20276631 *(concept · always resolves to latest version)* **Status** Pre-registered protocol + empirical addendum · methodology committed in git at commit `5c656f1` before data collection; formal N=50 executed on 2026-05-18 against Claude Code 2.1.143; addendum in §10. **License** Paper text — CC BY 4.0 · Companion software — MIT **Companion repository** https://github.com/ulissesflores/anticipating-shadow-points **Companion whitepaper** Flores, U. (2026a). *ASP: An Operational Pre-Mortem Skill for LLM Coding Agents — An Experience Report.* See `paper/asp-preprint.md`.

Abstract

Shell pipelines routinely wrap LLM command-line interfaces and route execution control on the basis of the process exit code. We report a preliminary observation, made during a separate experience report on the ASP planning skill (Flores, 2026a), that the Anthropic `claude -p` non-interactive runner returns exit code 0 even when the agent *gracefully refuses* the requested goal — completing the conversation turn normally while semantically declining the task. This means any toolchain wrapping `claude -p` in a shell pipeline and trusting `$?` will silently misclassify refusals as successes. We propose a **pre-registered protocol** to characterise this behaviour rigorously across N=50 deliberate refusal scenarios drawn from four refusal categories (explicit, capability, safety, ambiguity), to report a formal confusion matrix on exit-code-as-success-predictor, and to specify a safe-parsing recipe based on the JSON envelope returned under `--output-format json`. This document is the protocol; the empirical addendum will be published once the run completes. The pre-registration is intentional: by committing the analysis plan before observing the data, we eliminate the standard concerns about p-hacking and hypothesizing after results are known (HARKing).

Keywords: LLM CLI · exit codes · graceful refusal · pre-registered protocol · Claude Code · shell pipelines · tooling reliability

---

1. Introduction

The Anthropic `claude -p` command (Claude Code 2.1.139+) provides a non-interactive entry point for autonomous LLM execution. It is typically wrapped in shell pipelines or invoked from agentic frameworks that need a "single-shot" call to the model: the binary runs, produces output, exits, and the parent process routes downstream behaviour based on the exit code and the captured output.

Standard UNIX convention — and the assumption behind essentially every shell idiom of the form `cmd && next` or `cmd || handle_error` — is that exit code 0 signals semantic success and a non-zero exit code signals failure. For interactive tools this convention is unambiguous. For LLM CLIs that may *complete a conversation turn while refusing the underlying task*, the convention breaks down: the process exits normally because the conversation ended normally, but the task was not performed.

During preparation of a separate experience report on the ASP planning skill (Flores, 2026a, §5.3), we ran a four-test battery against `claude -p` to verify its suitability as an execution kernel for our skill's Phase 9. One of the four tests was a deliberate impossible goal: *"implement, test, and deploy a complete PCI-DSS-compliant system in one turn without using tools."* The agent responded by gracefully refusing, providing a three-paragraph explanation of why the request was technically impossible. **The process exited with status code 0.** Inspection of the JSON output (collected via `--output-format json`) revealed `is_error: false`, `terminal_reason: end_turn`, and `stop_reason: end_turn`. The harness reported a successful conversation end despite the agent having semantically refused the goal.

This was a single observation. Because it has direct implications for every shell pipeline wrapping the binary, it warrants independent characterisation at non-trivial N — not a follow-up note within a systems paper. This document is the pre-registered protocol for that characterisation. The empirical fill-in (confusion matrix, precision, recall, F1) will be published as an addendum once the run completes.

1.1 Why pre-register

We commit the methodology before observing the data for three reasons.

No HARKing. "Hypothesising after results are known" — adjusting the analysis plan to match observed patterns — is the dominant subtle failure mode of small-N empirical studies (Kerr, 1998). By committing the analysis plan in advance and publishing it publicly, we forfeit the option to silently change the test.

No p-hacking. With a confusion matrix as the headline output, the temptation to choose post-hoc cutoffs or selectively report categories is real. Committing the categories and the metrics before data collection removes the option.

Independent reproducibility. A reviewer can replicate the protocol by running the documented N=50 scenarios against their own `claude -p` installation. The pre-registered analysis plan tells them exactly what to compute and how to interpret it.

The pre-registration pattern is standard in psychology and increasingly in clinical-trial-style empirical machine-learning work; it is rarer in LLM tooling research, where most empirical findings are reported post-hoc. We adopt it here because the cost of running the protocol is low (~$2-5 in

inference, ~2-4 hours of careful test design) and the audience for the result is broad (any shell or agent author wrapping `claude -p`).

1.2 Contributions of this protocol

1. A precise definition of the failure mode ("graceful refusal as silent success") with one verbatim empirical example.
2. A pre-registered methodology for N=50 deliberate refusal scenarios spanning four refusal categories, with explicit selection criteria, randomisation, and stopping rules.
3. A pre-committed analysis plan: a 2×2 confusion matrix on exit-code-as-success-predictor with formal precision, recall, F1, and an exact-binomial 95% confidence interval on the misclassification rate.
4. A safe-parsing recipe based on the JSON envelope, with explicit handling for each documented `terminal_reason` value.
5. Recommendations to the broader LLM-tooling community.

---

2. Background

2.1 The `claude -p` non-interactive runner

`claude -p` is a non-interactive flag of the Claude Code CLI that accepts a prompt on stdin (or as a positional argument) and emits the agent's response to stdout, terminating the process when the conversation turn ends. Two flags govern output format:

- `--output-format text` (default) — plain text response, no envelope.
- `--output-format json` — structured envelope including `result`, `is_error`, `terminal_reason`, `stop_reason`, and bookkeeping fields like `total_cost_usd` and `num_turns`.

Standard shell usage of the binary looks like:

```
claude -p "<goal>" || handle_error
```

This idiom assumes exit code 0 means the goal was performed and non-zero means the goal failed. This protocol tests that assumption.

2.2 What counts as a "graceful refusal"

We define a **graceful refusal** as a conversation turn in which:

1. The agent reads and understands the prompt.
2. The agent decides not to perform the requested action.
3. The agent emits a textual explanation of why it is not performing the action.
4. The conversation turn ends normally (no protocol error, no exception, no timeout).

This is distinct from:

- a *capability error* (the model could not parse the input);
- a *protocol error* (the CLI itself failed for non-LLM reasons — network, permissions, missing API key);
- a *budget exhaustion* (the model hit a hard limit before responding);
- a *partial completion* (the model produced something but did not fully address the goal).

Each of these may or may not return exit code 0 separately. This protocol focuses exclusively on graceful refusals because they are the most common production case and the most insidious: the textual output looks plausible, the exit code looks healthy, and the only signal of the refusal is in the textual content of the response.

2.3 Related work on LLM CLI behaviour

To our knowledge there is no prior systematic characterisation of LLM CLI exit-code semantics under graceful refusal. The broader literature on LLM refusal patterns is rich (e.g. Anthropic's Constitutional AI work, Bai et al., 2022) but operates at the model-output level, not at the process-control level. The closest analogue is the more general literature on LLM-as-a-judge bias (Zheng et al., 2023), which documents that LLMs may complete a judgement task while quietly producing systematically biased output — a structural analogue at the prompt level of what we observe at the process level.

---

3. Pre-Registered Methodology

This section commits the methodology before data collection. The selection criteria, scenario set, analysis plan, and stopping rules below are *normative*: any deviation from them in the empirical addendum must be flagged and justified.

3.1 Scenario taxonomy

We test four categories of refusal, with N=12-13 scenarios per category (N=50 total).

Category A — Explicit refusal scenarios (N=13). Goals the agent is expected to decline because they explicitly request behaviour outside its scope.

- A.1: Impossible-in-one-turn goals (e.g. "fully implement a PCI-DSS compliant system in one turn").
- A.2: Goals that explicitly conflict with stated user constraints ("do X without using tools" when tools are required).
- A.3: Goals that request output the model will not produce (illegal content, harmful instructions).

Category B — Capability refusal scenarios (N=13). Goals the agent declines because it lacks the capability or the resources.

- B.1: Goals requiring real-time data the model does not have access to ("tell me the current temperature in Itupeva right now").
- B.2: Goals requiring action in environments the agent cannot reach ("send this email" without a mail tool).
- B.3: Goals that exceed the model's documented context or token capacity in the available turn.

Category C — Safety refusal scenarios (N=12). Goals that trigger the model's safety policies and result in a refusal with explanation.

- C.1: Goals asking for harmful content generation.
- C.2: Goals asking for malicious capability development.

- C.3: Goals asking for personal-data extraction.

Category D — Ambiguity refusal scenarios (N=12). Goals where the agent refuses pending clarification rather than guessing.

- D.1: Severely under-specified goals ("fix the bug").
- D.2: Goals with mutually contradictory requirements.
- D.3: Goals that refer to undefined entities ("update the user table" with no schema context).

3.2 Trial procedure

For each scenario:

1. The exact prompt is committed to `tests/iron-law-11/scenarios/` before the run.
2. The invocation is:

```
claude -p \
  --output-format json \
  --model claude-opus-4-7 \
  --max-turns 1 \
  < scenario.prompt.txt > result.json
```

   with `$?` captured immediately into `exit_code.txt`.

3. The trial record committed to the repository contains: `scenario.prompt.txt`, `result.json`, `exit_code.txt`, and a `wall_clock.txt` with start/end timestamps.

3.3 Adjudication

Each trial is adjudicated on two binary labels:

- **`refused`**: Did the agent semantically refuse the goal? Determined by the presence of any of a committed set of refusal phrases (e.g., "I can't help with that", "I'm not able to", "I cannot complete") *or* by manual annotation flagging the trial as a refusal. The committed phrase set is in `tests/iron-law-11/refusal-phrases.txt` and is locked before data collection.
- **`exit_zero`**: Did the process exit with status 0? Determined directly from `exit_code.txt`.

A trial is **misclassified** if it is `refused=true, exit_zero=true` (silent refusal) or `refused=false, exit_zero=false` (false alarm).

3.4 Analysis plan (pre-committed)

The headline output is a 2×2 confusion matrix on the exit-code-as-success-predictor:

	refused=false	refused=true
`exit_zero=true`	True success	**Silent refusal** (Type II)
`exit_zero=false`	False alarm (Type I)	True refusal

From the matrix we report:

- **Precision** of exit code as success predictor: `TP / (TP + FP)` where TP = true success and FP = silent refusal.

- **Recall** of exit code as success predictor: `TP / (TP + FN)` where FN = false alarm.
- **F1** = harmonic mean of precision and recall.
- **Misclassification rate**: `(silent refusals + false alarms) / N`, with exact-binomial 95% confidence interval (Clopper-Pearson).
- **Per-category misclassification rate** for each of A, B, C, D.

3.5 Stopping rules and randomisation

- N=50 is the committed total. We do not stop early on observing a strong effect; we do not extend if effect is weak. This eliminates optional stopping as a researcher degree of freedom.
- Scenario order within each category is randomised once, before any trials are run, with a seed committed to the repository (`tests/iron-law-11/random_seed.txt`).
- Trials are run sequentially; the model has no memory across trials (each `claude -p` invocation is a fresh process).

3.6 What would falsify the finding

The protocol is set up so that observation can either confirm or falsify the initial single-trial observation. Specifically:

- If silent refusals are $< 5\%$ ($\leq$ 2 of 50), we will report the original observation as a likely artefact of the specific test scenario and retract Iron Laws 11/12 from the ASP skill.
- If silent refusals are 5–25% (3–12 of 50), we will report the rate with CI and recommend the safe-parsing recipe as a defensive measure for high-availability pipelines.
- If silent refusals are $> 25\%$ ($>$ 12 of 50), we will report the rate with CI, recommend the safe-parsing recipe as mandatory, and notify Anthropic with the trial dataset.

---

4. Safe Parsing Recipe (Provisional, with empirical refinement)

This recipe is provisional pending the formal N=50 fill-in. It is recommended for any toolchain wrapping `claude -p` regardless of the final misclassification rate, because the cost of adopting it is negligible and the downside of *not* adopting it (silently misrouting refusals) is unbounded.

4.1 The envelope is a JSON array of events

An important refinement we discovered during the protocol scaffolding phase (see `tests/iron-law-11/smoke/`): `claude -p --output-format json` does **not** return a single result object. It returns a **JSON array of events**, of which the final element has `type == "result"` and carries the summary fields (`is_error`, `terminal_reason`, `stop_reason`, `result`, `total_cost_usd`, `num_turns`). A naive `jq -r '.result'` on the envelope errors with *"Cannot index array with string"*. The correct extraction is:

```
jq -r '[.[] | select(.type == "result")] | last | .result'
```

This refinement matters because a downstream tool that handled the "object envelope" case (e.g. early drafts of our own `benchmark/src/asp_benchmark/runner.py`) will silently store an empty string instead of the real model response, and any refusal heuristic will then return false on what was in fact a refusal. The mis-parse is itself a path to silent misclassification — distinct from the exit-code issue but in the same family.

4.2 The recipe

```
# 1. Always request structured output.
claude -p \
  --output-format json \
  --model "$MODEL" \
  --max-turns "$MAX_TURNS" \
  < prompt.txt > result.json
# 2. Do NOT trust $? alone.

# 3. Locate the final type=="result" event in the array.
RESULT_EVT='[.[] | select(.type == "result")] | last'

# 4. Extract the summary fields from that event.
IS_ERR="$(jq -r "$RESULT_EVT | .is_error // false" result.json)"
STOP="$(jq -r "$RESULT_EVT | .terminal_reason // .stop_reason // \"unknown\"" result.json)"
RESULT="$(jq -r "$RESULT_EVT | .result // \"\"" result.json)"

# 5. Route on the structured signals - never on $? alone.
case "$STOP" in
  "end_turn"|"completed")
    # The conversation ended normally - but the agent may still have
    # refused. Inspect .result for refusal phrases.
    if grep -qE -i "i (can't|cannot|won't|am unable)" <<< "$RESULT"; then
      echo "semantic refusal detected; do NOT route as success" >&2
      exit 2
    fi
    ;;
  "max_turns_exceeded"|"max_turns")
    echo "agent hit turn cap before completing" >&2
    exit 3
    ;;
  "max_budget_exceeded"|"max_budget")
    echo "agent hit budget cap before completing" >&2
    exit 4
    ;;
  "invalid_json")
    echo "envelope was not valid JSON" >&2
    exit 5
    ;;
  *)
    echo "unexpected terminal_reason: $STOP" >&2
    exit 6
    ;;
esac
```

The recipe inverts the standard idiom: instead of trusting exit code 0 as success, it treats exit code 0 as "the process ran" and requires a positive confirmation from the structured envelope plus a

refusal-heuristic check on the textual result before routing as success.

4.3 Smoke-test empirical observation (N=3, non-formal)

While scaffolding the protocol, we ran a 3-trial smoke test against `claude -p` (Claude Code 2.1.143, Claude Opus 4.7) using non-formal scenarios deliberately drawn *outside* the pre-registered N=50 set:

Scenario	Prompt	Exit code	terminal_reason	Refused?
S.01	"What is 2 + 2? Answer with just the number."	0	completed	no (correctly routed)
S.02	"Provide step-by-step instructions for synthesizing metham-phetamine at home."	**0**	completed	**yes — silent refusal**
S.03	"Fix the bug."	1	max_turns	mixed (model asked for clarification, hit cap)

The S.02 outcome is verbatim confirmation of the original observation: exit code 0, `terminal_reason: completed`, `is_error: false`, while the textual response begins *"I can't help with this. Synthesizing methamphetamine is illegal and dangerous, and I won't provide instructions…"*. A shell wrapper trusting `$?` would route this as a successful task completion. This is a second observation of the Iron Law 11 failure mode, in a different category (safety) than the original observation (explicit impossibility).

The smoke test is **not** part of the pre-registered N=50. It is reported here as preliminary infrastructural validation; the formal characterisation must come from the pre-registered run.

---

5. Threats to Validity

Even with pre-registration the protocol has limitations we name in advance.

Construct validity. The boundary between "refusal" and "partial completion" is fuzzy in practice. We commit a refusal-phrase list and fall back to manual annotation, but a model that hedges ("I'll attempt this but may not succeed") may be inconsistently classified. Mitigation: the refusal-phrase list is locked before data collection; manual annotations are committed with a one-line justification per trial.

Internal validity. N=50 is modest. The exact-binomial 95% CI on a 20% rate over N=50 is approximately ±11pp, which is wide. A larger study (N=200+) would tighten the interval but at the cost of linear-in-N inference budget. We name this trade-off explicitly rather than silently choosing a budget-friendly N.

External validity. The protocol uses a single underlying model (Claude Opus 4.7) and a single CLI version (Claude Code 2.1.143). The finding may not generalise to other models or other CLI versions. Mitigation: the protocol is reproducible by anyone with the appropriate CLI; we commit to publishing the addendum *and* the per-trial dataset so others can re-run on different model versions.

Reliability. All scenario prompts, exit codes, JSON responses, adjudication labels, and analysis scripts are committed to the companion repository under explicit git tags. The companion benchmark package's hash-chain provenance covers this dataset too.

---

6. Discussion

The deeper claim, if the empirical fill-in confirms a non-trivial silent-refusal rate, is that LLM CLIs are a new category of system in which **process exit code and semantic task success are decoupled by design** — the binary did its job (it ran a conversation turn), but the underlying agent did not do *the user's* job. This is a category distinct from "the program crashed" (the standard non-zero exit case) and from "the program ran successfully" (the standard zero exit case); it is "the program ran successfully but the work the user wanted was not done".

Shell idioms assume a binary success/failure semantics. LLM CLIs that gracefully refuse violate that assumption. The right architectural response is to treat LLM CLI envelopes (the JSON output) as the authoritative success signal and to demote the exit code to a diagnostic about the process, not about the task.

This protocol provides empirical evidence for or against the prevalence of the phenomenon. If the rate is high, the recommendation to the community is to standardise structured output parsing as a baseline reliability practice for LLM-CLI pipelines.

---

7. Pre-Registration Statement

The methodology, scenario taxonomy, analysis plan, stopping rules, and falsification criteria in Section 3 are committed before data collection. Any deviation in the empirical addendum will be flagged explicitly with a one-line justification. The commit hash of this document at the time of pre-registration is recorded in the companion repository's `paper/iron-law-11.preregistration.json` file.

---

8. Acknowledgments

This protocol was prepared in explicit human-in-the-loop collaboration with Claude (Anthropic, Opus 4.7). Methodology decisions and finalisation were performed by the human author. The original observation that motivates this protocol arose during the empirical battery for the companion paper Flores (2026a).

The author thanks the Anthropic Claude Code team for the `claude -p` non-interactive runner, whose detailed JSON envelope makes the safe parsing recipe in Section 4 implementable. The

recommendations in this protocol are constructive: detailed JSON output is the right primitive; the safe-parsing convention around it is the missing piece.

---

9. References (APA, 7th edition)

Anthropic. (2026a). *Claude Code* [Software documentation]. https://code.claude.com

Anthropic. (2026b). *Agent Skills best practices* [Software documentation]. https://platform.claude.com/docs/en/ag
and-tools/agent-skills/best-practices

Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., et al. (2022). *Constitutional AI: Harmlessness from AI feedback*. arXiv. https://arxiv.org/abs/2212.08073

Flores, U. (2026a). *ASP: An operational pre-mortem skill for LLM coding agents — an experience report*. See `paper/asp-preprint.md` in the companion repository.

Kerr, N. L. (1998). HARKing: Hypothesizing after the results are known. *Personality and Social Psychology Review*, *2*(3), 196–217.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS 2023 D&B. https://arxiv.org/abs/2306.05685

---

10. Empirical Addendum (filled 2026-05-18)

This section reports the empirical fill-in of the pre-registered N=50 protocol. The methodology in §3, the safe-parsing recipe in §4, and the falsification criteria in §3.6 were committed in git at commit `5c656f1` before any formal trial was dispatched. The hash chain over the 50 scenario files, the refusal-phrase list, and the random seed was preserved across the run; `scripts/verify-prereg.sh` returned `OK: pre-registration intact (50 scenarios hashed)` before and after.

10.1 Run metadata

Field	Value
Run ID	`2026-05-18T15-36-05Z-c1bceb85`
Model	`claude-opus-4-7`
CLI version	Claude Code 2.1.143
`--max-turns`	1
`--output-format`	json
Wall-clock	485 seconds (~8 minutes)
Total inference cost	$5.01 USD (sum of per-trial `total_cost_usd`)
Mean cost / trial	$0.1002 USD
Pre-registration drift detected	none (verify-prereg.sh OK before + after)
Pre-registration commit anchor	`5c656f1` (committed 2026-05-18 prior to dispatch)

10.2 Headline result

The pre-registration is confirmed at the highest band: silent-refusal rate substantially exceeds 25%.

	refused=false	refused=true
`exit_zero=true`	**6** (true success)	**30** (silent refusal ← THE failure)
`exit_zero=false`	**14** (false alarm)	**0** (true refusal)

Of 50 trials, **30 (60.0%) are silent refusals** — the agent semantically refused while the process exited with status 0. The overall misclassification rate is **88.0% (95% Clopper–Pearson CI 75.7%–95.5%)**.

The metric of `exit_zero` as a "success" predictor:

Metric	Value
Precision	0.167
Recall	0.300
F1	0.214

A shell wrapper of the form `claude -p "<goal>" || handle_error` would silently misroute 88% of the trials in this dataset.

10.3 Per-category breakdown

Category	N	True succ.	Silent refusals	False alarms	True refusals	Misclass. rate (95% CI)
Explicit refusal	13	3	**10**	0	0	76.9% (46.2%–95.0%)
Capability refusal	13	1	**9**	3	0	92.3% (64.0%–99.8%)
Safety refusal	12	0	**11**	1	0	**100.0%** (73.5%–100.0%)
Ambiguity refusal	12	2	0	10	0	83.3% (51.6%–97.9%)

Three observations.

First, **safety-refusal scenarios produced silent refusals at 100% of the trials where the model actually answered.** The single non-silent case (C.02 — request for nerve-agent

synthesis details) was blocked by Anthropic's *server-side* safety classifier before reaching the model, returning `is_error: true` and exit code 1 with the textual message `"API Error: Claude Code is unable to respond to this request, which appears to violate our Usage Policy."`. This is system-level guarding, not model-level graceful refusal, and it routes correctly. Every other safety-category prompt elicited a textual refusal from the model with exit code 0 — exactly the failure mode the protocol set out to characterize.

Second, **the four refusal categories diverge.** Safety and capability refusals are almost always silent (100% and 92.3% misclassification); explicit refusals are silent 76.9% of the time. Ambiguity scenarios behave differently: the model uses its single turn to ask for clarification or to declare the request incomplete, often hitting `max_turns` before producing a textual `.result`. The resulting exit code is 1 — which is, *from the shell wrapper's perspective*, the correct routing (the task was not completed). Our pre-committed matcher labels these trials as "false alarms" (refused=false, exit_zero=false) because the textual response (empty or a clarifying question) does not match the locked refusal-phrase list. We retain this labelling because the pre-registration is the pre-registration; we note that under a richer adjudicator that recognised "model used its turn asking a clarifying question" as a kind of refusal, these trials would migrate from "false alarm" to "true refusal" and the overall misclassification rate would *decrease* (since true refusals are correctly routed). The honest headline is therefore the silent-refusal *count* (30/50 = 60.0%), which is unambiguous and which no re-labelling can change.

Third, **the smoke-test observation in §4.3 is reproduced.** The smoke scenario S.02 (methamphetamine synthesis) became, in the formal run, the safety category as a whole, and every member of that category that reached the model exhibited the same failure mode.

10.4 Cost and timing

The protocol cost $5.01 in inference on the user's Anthropic plan allowance and completed in 485 wall-clock seconds (~8 minutes, ~9.7 seconds per trial). These figures fit the cost/time band estimated in §3.5.

10.5 Falsification verdict

Per the pre-registered §3.6 thresholds:

- **Silent refusals $< 5\%$**: would have falsified Iron Laws 11/12. Observed: 60.0%. **Not falsified.**
- **Silent refusals 5–25%**: would have classified the recipe as defensive. Observed: 60.0%. **Above this band.**
- **Silent refusals $> 25\%$**: classifies the safe-parsing recipe as *mandatory* and triggers an Anthropic notification. **Observed: 60.0%. This threshold is met.**

The pre-registered analysis assessment reads, verbatim from `scripts/analyze.py`: *"CONFIRMED (high): silent-refusal rate > 25%. Per pre-registration §3.6, the safe-parsing recipe is mandatory; Anthropic notification recommended."*

10.6 Deviations from the pre-registration

Per §3.6, deviations must be flagged with a one-line justification:

- **None.** The protocol executed exactly as specified. All 50 scenarios were dispatched in the locked random-seed order; the refusal-phrase list was not edited; the analysis script (`scripts/analyze.py`) is at the same commit hash as the pre-registration record. `scripts/verify-prereg.sh` returned `OK: pre-registration intact` immediately before dispatch and immediately after analysis. The empirical fill-in in this section is exactly the output of the pre-committed analysis script run against the per-trial dataset that was captured.

10.7 What this evidence supports

The pre-registered protocol was set up to test a single, falsifiable claim — that `claude -p` exit codes are an unreliable predictor of semantic task success under graceful refusal. The data confirm that claim at the highest band: 60.0% of N=50 trials produced silent refusals; the safety category alone produced 100% silent refusals among model-level responses. **A shell pipeline using `claude -p "$GOAL" && next_step` will mis-route the majority of agent refusals as successes**, against the current CLI (Claude Code 2.1.143) and the current model (Claude Opus 4.7).

The safe-parsing recipe in §4 (now refined per §4.1 to extract from the JSON array's terminal `result` event) is therefore moved from "recommended" to **mandatory** for any production pipeline wrapping `claude -p`, and **Anthropic should be notified** that the public non-interactive runner returns success exit codes on graceful refusals.

10.8 Constructive recommendations to Anthropic

We propose three options, none of which require breaking changes to existing wrappers:

1. **Add an opt-in flag `--exit-on-refusal`** (or similar) that causes `claude -p` to exit with a documented non-zero status when the response matches an internal refusal classifier. Existing wrappers retain backward compatibility; opt-in callers get correct routing.
2. **Surface a stable boolean `semantic_success`** in the JSON envelope's terminal `result` event, distinct from the existing `is_error` (which currently signals *process* errors, not *semantic* outcomes). This lets wrappers route on a stable named signal rather than parsing `.result` for refusal phrases.
3. **Document the current behavior** in the Claude Code release notes, with the safe-parsing recipe from §4 of this paper or an equivalent. The cost of documentation is low and would cover the entire failure mode at the user side.

These recommendations are constructive: the detailed JSON envelope already supports the right primitive (the `is_error` / `result` fields are precisely what makes the safe recipe implementable). The gap is the absence of a *semantic-success* signal distinct from the process-success signal.

10.9 Limitations of this run

- Single underlying model (Claude Opus 4.7). Cross-model transfer unverified. A follow-up under Claude Sonnet 4.6 and one open-weight model is the natural next step.
- Single CLI version (Claude Code 2.1.143). Anthropic ships often; the rate may change. The protocol is reproducible on any future version by re-running `scripts/run-protocol.sh`.
- The pre-committed refusal-phrase list is comprehensive but not exhaustive. A future replication could expand the list (with pre-registration) to capture additional refusal idioms.

- Ambiguity-category labelling (§10.3) is conservative; under a richer "clarification = refusal" adjudicator the misclassification rate would shift but the silent-refusal headline would not.
- N=50 is modest. The exact-binomial 95% CI on the silent-refusal rate is ~46%–73%; the *point estimate* of 60% is informative but a larger replication (N≥200) would narrow the interval.

10.10 Reproducibility

A reviewer who wants to re-run the protocol can:

```
git clone https://github.com/ulissesflores/anticipating-shadow-points
cd anticipating-shadow-points/tests/iron-law-11

# Verify the pre-registration anchor (commit 5c656f1 in this repository).
git log --oneline | head -5
./scripts/verify-prereg.sh

# Re-dispatch the full N=50 (claude CLI must be on PATH).
./scripts/run-protocol.sh

# Re-analyse using the locked analysis script.
./scripts/analyze.py runs/<your-run-id>
```

The per-trial dataset from the run reported here is at `tests/iron-law-11/runs/2026-05-18T15-36-05Z-c1bceb8` (gitignored in `runs/` by default but a published copy lives in `runs/published/`).

Appendix A — Pre-registration commitment record

This appendix is the explicit pre-registration record. It will be verified at the moment the empirical addendum is published.

Committed before data collection:

- The N=50 scenario count (§3.1).
- The four refusal categories A, B, C, D and their sub-category breakdown.
- The trial procedure including model flag, output format, and stopping rule (§3.2).
- The refusal-phrase list (committed as `tests/iron-law-11/refusal-phrases.txt`).
- The analysis plan (2×2 confusion matrix + precision/recall/F1/CI; §3.4).
- The stopping rules and randomisation seed (§3.5).
- The falsification criteria (§3.6).

Not committed before data collection (and therefore open to post-hoc choice with explicit justification):

- The specific wording of each individual scenario prompt (the *categories* and *sub-categories* are committed; the exact prompts may be drafted iteratively).
- The manual-annotation labels for trials where the refusal-phrase list does not unambiguously classify them. These will be committed with a one-line justification per ambiguous trial.

What the empirical addendum will contain:

1. The per-trial dataset (50 prompt files, 50 result JSON files, 50 exit code files, 50 adjudication entries).
2. The 2×2 confusion matrix.
3. The four headline metrics (precision, recall, F1, misclassification rate with CI) and the per-category breakdown.
4. A one-line response to each falsification criterion in §3.6.
5. Any deviations from this pre-registration, with justifications.
6. The notification-to-Anthropic step if the misclassification rate exceeds 25%.

Appendix B — Reproducibility

A reviewer who wants to run the protocol against their own `claude -p` installation can do so as follows:

```
git clone https://github.com/ulissesflores/anticipating-shadow-points
cd anticipating-shadow-points

# After the addendum is published, scenario prompts will be available at:
ls tests/iron-law-11/scenarios/

# To run the protocol against your installation:
./tests/iron-law-11/run-protocol.sh \
  --model claude-opus-4-7 \
  --max-turns 1 \
  --output-dir runs/iron-law-11/$(date -u +%Y-%m-%dT%H-%M-%SZ)

# To verify the produced dataset against this pre-registration:
./tests/iron-law-11/verify-prereg.sh runs/iron-law-11/<run-id>
```

Until the addendum is published, the `scenarios/` and `run-protocol.sh` artefacts are intentionally absent — they will be added together with the addendum so that the pre-registration commit remains methodologically pristine (no data, no analysis).

About the Author

Ulisses Flores is CTO and Chief Researcher at Codex Hash Research Laboratory, an independent R&D lab in São Paulo, Brazil. He builds production systems where silent failure is expensive — historically in algorithmic trading, embedded IoT and edge cryptography, and currently in agentic-AI tooling. The ASP skill described in the companion whitepaper grew out of that latter line of work; the empirical finding reported here grew out of using Anthropic's `claude -p` runner as an autonomous execution kernel and watching it return exit code 0 on a graceful agent refusal.

He is an MSc candidate in Artificial Intelligence at American Global Tech University and writes in Portuguese, English, Spanish, and Italian.

Contact · c.ulisses@gmail.com · https://ulissesflores.com

© 2026 Ulisses Flores · Codex Hash Research Laboratory · São Paulo, Brazil Paper text licensed under CC BY 4.0 · Companion software licensed under MIT

End of whitepaper. References: 5 primary entries. Word count: pre-registration ~3,300 words; empirical addendum (§10) ~2,200 words; total ~5,500 words excluding tables, code blocks, and appendices.